PacketProcessor.dll

API 사용 가이드 · v1.0 · 배포용 라이브러리 문서

1. 배포 파일 구성

파일	용도
`PacketProcessor.dll`	런타임 바이너리 — 실행 파일과 같은 폴더에 위치
`PacketProcessor.h`	C/C++ 헤더 — 구조체·콜백·API 선언 포함
`PacketProcessor.lib`	C/C++ 임포트 라이브러리 — 링커에 전달

> **C# / .NET 환경**에서는 `.lib` 파일 없이 `PacketProcessor.dll` 과 P/Invoke 선언만으로 사용합니다.

2. 사용 흐름 (라이프사이클)

`Create` → `Start` → `Enqueue ×N` → `Reset (선택)` → `Stop` → `Destroy`

단계	설명
`Create`	설정과 콜백을 등록하여 인스턴스를 생성한다
`Start`	내부 워커 스레드를 시작한다. Enqueue 전 필수
`Enqueue`	캡처된 TCP 페이로드를 투입한다. Thread-safe
`Reset`	전투 포트 락·스트림 상태만 초기화. 스레드는 유지
`Stop`	워커 스레드를 종료하고 큐를 플러시한다
`Destroy`	인스턴스를 해제한다 (내부에서 Stop 자동 호출)

3. 설정 구조체 — `PacketProcessorConfig`

```
typedef struct PacketProcessorConfig {
    int serverPort;       // 서버 포트 (0 = 자동 감지)
    int tcpReorder;       // TCP 시퀀스 재조립 (1=활성화, 0=비활성화)
    int workerCount;      // 워커 스레드 수 (0 = 자동: max(2, CPU/2))
    int maxBufferSize;    // 스트림 버퍼 바이트 (0 = 기본값 2MB)
    int maxReorderBytes;  // TCP 재조립 버퍼 (0 = 기본값 128KB)
} PacketProcessorConfig;
```

> 모든 필드를 0으로 설정하면 권장 기본값으로 동작한다.

4. 콜백 구조체 — `PacketCallbacks`

```
typedef struct PacketCallbacks {
    // ── 이벤트 (DLL → 호스트) ──────────────────
    OnDamageCallback          onDamage;
    OnMobSpawnCallback        onMobSpawn;
    OnSummonCallback          onSummon;
    OnUserInfoCallback        onUserInfo;
    OnEntityRemovedCallback   onEntityRemoved;
    OnLogCallback             onLog;

    // ── 쿼리 (DLL이 호스트에게 데이터를 물어봄) ──
    GetSkillNameCallback      getSkillName;
    ContainsSkillCodeCallback containsSkillCode;
    IsMobBossCallback         isMobBoss;

    void*                     userdata; // 모든 콜백에 그대로 전달
} PacketCallbacks;
```

> 모든 콜백은 **DLL 내부 워커 스레드**에서 호출된다. 호스트 측 데이터 접근 시 동기화가 필요하다. 불필요한 콜백은 `NULL` 로 두어도 된다.

4-1. 이벤트 콜백 시그니처

```
// 대미지 발생
typedef void (*OnDamageCallback)(
    int      actorId,        // 공격자 엔티티 ID
    int      targetId,       // 피격자 엔티티 ID
    int      skillCode,      // 스킬 코드
    uint8_t  damageType,     // 0=일반, 3=크리티컬
    int      damage,         // 최종 대미지 (멀티힛 제외)
    uint32_t specialFlags,   // 특수 효과 비트마스크
    int      multiHitCount,  // 멀티힛 횟수 (0=단일)
    int      multiHitDamage, // 멀티힛 합산 대미지
    int      healAmount,     // 회복량
    int      isDot,          // 1=DoT, 0=즉발
    void*    userdata);

// 몹 스폰 (코드 확정 시 같은 mobId로 재호출될 수 있음)
typedef void (*OnMobSpawnCallback)(
    int mobId, int mobCode, int hp, int isBoss, void* userdata);

// 소환수 등장
typedef void (*OnSummonCallback)(
    int actorId, int petId, void* userdata);

// 유저 정보 (nickname: UTF-8, "닉네임[서버]" 형식 가능)
typedef void (*OnUserInfoCallback)(
    int entityId, const char* nickname,
    int serverId, int jobCode, void* userdata);

// 엔티티 제거
typedef void (*OnEntityRemovedCallback)(
    int entityId, void* userdata);

// DLL 내부 로그 (0=INFO, 1=WARN, 2=ERROR / Debug 빌드에서만 상세 출력)
typedef void (*OnLogCallback)(
    int level, const char* message, void* userdata);
```

4-2. specialFlags 비트마스크

상수	값	의미
SPECIAL_BACK	0x0001	백어택
SPECIAL_SHIELD_BLOCK	0x0002	방패 막기
SPECIAL_WEAPON_BLOCK / PARRY	0x0004	막기 / 패리
SPECIAL_PERFECT	0x0008	퍼펙트 블록
SPECIAL_HARD_HIT / DOUBLE	0x0010	강타
SPECIAL_IRON_WALL	0x0020	철벽
SPECIAL_SMITE / RESTORATION	0x0040	스마이트 / 리스토레이션
SPECIAL_POWER_SHARD	0x0080	파워 샤드
SPECIAL_CRITICAL	0x0100	크리티컬 (damageType==3이면 자동 설정)

4-3. 쿼리 콜백 시그니처

```
// 스킬 이름 반환. 없으면 NULL. 반환 포인터는 다음 동일 스레드 호출 전까지 유효해야 함
typedef const char* (*GetSkillNameCallback)(int skillCode, void* userdata);

// 스킬 코드 존재 여부: 1=있음, 0=없음
typedef int (*ContainsSkillCodeCallback)(int skillCode, void* userdata);

// 보스 여부: 1=보스, 0=일반
typedef int (*IsMobBossCallback)(int mobCode, void* userdata);
```

5. API 함수 레퍼런스

PacketProcessor_Create

```
void* PacketProcessor_Create(
    const PacketProcessorConfig* config,
    const PacketCallbacks*       callbacks);
```

반환값	성공: 불투명 핸들 포인터 / 실패: NULL
비고	config와 callbacks는 내부적으로 복사된다

PacketProcessor_Enqueue Thread-safe

```
void PacketProcessor_Enqueue(
    void*          handle,
    int            srcPort,
    int            dstPort,
    const uint8_t* data,
    int            dataLen,
    const char*    deviceName,  // NULL 허용
    uint32_t       seqNum);
```

data	TCP 페이로드 바이트 배열. 내부에서 복사되므로 호출 후 해제 가능
deviceName	캡처 장치 이름 (ASCII/UTF-8). 멀티 인터페이스 환경에서 포트 감지에 사용
seqNum	TCP 시퀀스 번호. `tcpReorder=0` 이면 무시됨

PacketProcessor_GetCombatPort · PacketProcessor_GetCombatDevice

```
int         PacketProcessor_GetCombatPort  (void* handle); // 미감지: -1
const char* PacketProcessor_GetCombatDevice(void* handle); // 미감지: NULL
```

> 반환된 문자열 포인터는 핸들이 유효한 동안만 사용 가능하다.

PacketProcessor_Reset

```
void PacketProcessor_Reset(void* handle);
```

전투 포트 락·스트림 상태를 초기화한다. 워커 스레드는 계속 동작한다.

6. C++ 사용 예시

```cpp
#include "PacketProcessor.h"
#include <cstdio>

// ── 콜백 구현 ──────────────────────────────────────
void OnDamage(int actorId, int targetId, int skillCode,
              uint8_t dmgType, int damage, uint32_t flags,
              int multiCount, int multiDmg,
              int heal, int isDot, void* ud)
{
    printf("[DMG] %d → %d  skill=%d  dmg=%d%s\n",
           actorId, targetId, skillCode, damage,
           (flags & SPECIAL_CRITICAL) ? " [CRIT]" : "");
}

void OnLog(int level, const char* msg, void* ud) {
    const char* lv[] = { "INFO", "WARN", "ERROR" };
    printf("[%s] %s\n", lv[level < 3 ? level : 2], msg);
}

// ── 메인 ──────────────────────────────────────────
int main() {
    // 1. 설정
    PacketProcessorConfig cfg = {};  // 전부 0 = 기본값
    cfg.tcpReorder = 1;              // TCP 재조립 활성화

    // 2. 콜백 등록
    PacketCallbacks cbs = {};
    cbs.onDamage = OnDamage;
    cbs.onLog    = OnLog;

    // 3. 인스턴스 생성 및 시작
    void* handle = PacketProcessor_Create(&cfg, &cbs);
    if (!handle) { printf("Create failed\n"); return 1; }
    PacketProcessor_Start(handle);

    // 4. 패킷 투입 (캡처 루프에서 호출)
    // PacketProcessor_Enqueue(handle, srcPort, dstPort, data, len, "eth0", seqNum);

    // 5. 종료
    PacketProcessor_Stop(handle);
    PacketProcessor_Destroy(handle);
    return 0;
}
```

C++ 링커 설정 (Visual Studio)

항목	값
추가 포함 디렉터리	헤더 폴더 경로 (PacketProcessor.h 위치)
추가 라이브러리 디렉터리	PacketProcessor.lib 위치
추가 종속성	`PacketProcessor.lib`
런타임 DLL 위치	PacketProcessor.dll을 실행 파일과 같은 폴더에 복사

7. C# P/Invoke 사용 예시

```
using System;
using System.Runtime.InteropServices;
using System.Text;

public class PacketProcessorBridge : IDisposable
{
    private const string DLL = "PacketProcessor.dll";

    // ── Native 구조체 ──────────────────────────────
    [StructLayout(LayoutKind.Sequential)]
    private struct NativeConfig {
        public int serverPort, tcpReorder, workerCount,
                   maxBufferSize, maxReorderBytes;
    }

    [StructLayout(LayoutKind.Sequential)]
    private struct NativeCallbacks {
        public OnDamageDelegate   onDamage;
        public OnMobSpawnDelegate onMobSpawn;
        public OnLogDelegate      onLog;
        /* ... 나머지 콜백 ... */
        public IntPtr             userdata;
    }

    // ── 델리게이트 ────────────────────────────────
    [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
    private delegate void OnDamageDelegate(
        int actorId, int targetId, int skillCode,
        byte damageType, int damage, uint specialFlags,
        int multiHitCount, int multiHitDamage,
        int healAmount, int isDot, IntPtr ud);

    // nickname은 UTF-8이므로 IntPtr로 받아 PtrToStringUTF8 사용
    [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
    private delegate void OnUserInfoDelegate(
        int entityId, IntPtr nickname, int serverId,
        int jobCode, IntPtr ud);

    [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
    private delegate void OnLogDelegate(
        int level, IntPtr message, IntPtr ud);

    // ── DLL Import ────────────────────────────────
    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    private static extern IntPtr PacketProcessor_Create(
        ref NativeConfig cfg, ref NativeCallbacks cbs);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    private static extern void PacketProcessor_Destroy(IntPtr h);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    private static extern void PacketProcessor_Start(IntPtr h);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    private static extern void PacketProcessor_Enqueue(
        IntPtr h, int srcPort, int dstPort,
        [MarshalAs(UnmanagedType.LPArray, SizeParamIndex = 4)] byte[] data,
        int dataLen,
```

```
        [MarshalAs(UnmanagedType.LPStr)] string? deviceName,
        uint seqNum);

    // ── 사용 예시 ──────────────────────────────────
    private readonly IntPtr _handle;
    private readonly OnDamageDelegate _onDmg;  // GC 방지용 필드 보관 필수
    private readonly OnLogDelegate    _onLog;

    public PacketProcessorBridge() {
        _onDmg = (actor, target, skill, type, dmg, flags, mc, md, heal, dot, ud) =>
            Console.WriteLine($"[DMG] {actor}→{target} skill={skill} dmg={dmg}");

        _onLog = (level, msgPtr, ud) =>
            Console.WriteLine($"[DLL] {Marshal.PtrToStringUTF8(msgPtr)}");

        var cfg = new NativeConfig { tcpReorder = 1 };
        var cbs = new NativeCallbacks { onDamage = _onDmg, onLog = _onLog };

        _handle = PacketProcessor_Create(ref cfg, ref cbs);
        PacketProcessor_Start(_handle);
    }

    public void Feed(int src, int dst, byte[] data, uint seq)
        => PacketProcessor_Enqueue(_handle, src, dst, data, data.Length, null, seq);

    public void Dispose() => PacketProcessor_Destroy(_handle);
}
```

⚠ 델리게이트 GC 주의

콜백으로 전달한 델리게이트 객체는 **반드시 클래스 필드로 보관**해야 한다. 지역 변수로만 선언하면 GC가 수집하여 콜백 호출 시 크래시가 발생한다.

UTF-8 문자열 수신

`OnUserInfoCallback` 의 `nickname` 과 `OnLogCallback` 의 `message` 는 UTF-8 인코딩이다. C#에서는 `[MarshalAs(UnmanagedType.LPStr)]` 대신 `IntPtr` 로 받아 `Marshal.PtrToStringUTF8()` 로 변환해야 한글이 깨지지 않는다.

8. 주의 사항 요약

항목	내용
Thread-safe API	`Enqueue` 만 스레드 안전. 나머지는 단일 스레드에서 호출 권장
콜백 호출 스레드	모든 이벤트 콜백은 DLL 워커 스레드에서 호출됨. 공유 데이터는 lock 필요
문자열 인코딩	DLL 경계의 모든 문자열은 UTF-8. C#에서 ANSI(LPStr) 마샬링 사용 금지
nickname 포인터 수명	콜백 함수 내에서만 유효. 저장하려면 즉시 복사
GetSkillName 반환값	반환 포인터는 같은 스레드의 다음 호출 전까지 유효해야 함 (thread-static 버퍼 권장)
Destroy 후 핸들	Destroy 후 핸들 및 GetCombatDevice 반환값 사용 금지
Debug 로그	onLog 콜백의 상세 패킷 로그는 Debug 빌드에서만 발생
플랫폼	Windows x64 전용 (MSVC v143 빌드)

9. 예시 프로젝트 — SimpleMeter

`examples/SimpleMeter/` 는 PacketProcessor.dll API를 C#에서 사용하는 방법을 보여주는 콘솔 DPS 미터 예시 프로젝트입니다. WinDivert로 TCP 패킷을 캡처해 DLL에 투입하고, `onDamage` 콜백으로 실시간 대미지를 집계합니다.

9-1. 주요 클래스 구조

클래스	참조 원본	역할
`PacketProcessorNative`	`PacketProcessor.h` 1:1	P/Invoke 선언 전체
`PacketProcessorBridge`	`PacketProcessorBridge.cs` 간소화	DLL 래퍼, C# 이벤트로 노출
`WfpCapturer`	`WfpCapturer.cs` 간소화	WinDivert SNIFF 캡처 루프
`Program`	—	콘솔 미터 UI, R=리셋 Q=종료

9-2. serverPort 설정의 중요성

> ⚠ **serverPort = 0 사용 금지**
> `serverPort` 를 0(자동 감지)으로 설정하면 `StreamAssembler` 가 클라이언트→서버 방향과 서버→클라이언트 방향 패킷을 **동일한 TCP 재조립 버퍼에 혼합**합니다. 두 방향의 TCP 시퀀스 번호는 완전히 독립적이므로 스트림이 즉시 오염되어 파싱이 멈춥니다.
> **반드시 실제 게임 서버 포트를 지정하세요.**

```
// ✗ 잘못된 예 — 양방향이 하나의 버퍼에 섞임
var bridge = new PacketProcessorBridge(serverPort: 0);

// ✓ 올바른 예 — 서버→클라이언트(A)와 클라이언트→서버(B) 버퍼 분리
var bridge = new PacketProcessorBridge(serverPort: 13328);
```

9-3. SimpleMeter 핵심 흐름

```
// 1. Bridge 생성 (서버 포트 지정 필수)
var bridge = new PacketProcessorBridge(serverPort: 13328, tcpReorder: true);

// 2. 이벤트 구독
bridge.OnDamage += (actorId, targetId, skillCode, dmgType, damage, flags,
                    multiCount, multiDmg, heal, isDot) => {
    int total = damage + multiDmg;   // 멀티힛: damage=0, multiDmg=합산
    if (total > 0) RecordDamage(actorId, total, flags);
};
bridge.OnUserInfo += (entityId, name) => RegisterName(entityId, name);

// 3. WinDivert 캡처 시작
var capturer = new WfpCapturer(bridge);
capturer.Start();   // 내부에서 bridge.Start() 호출 후 캡처 루프 시작

// 4. 종료 시
capturer.Dispose();
bridge.Dispose();
```

9-4. 빌드 및 실행

단계	내용
사전 조건	관리자 권한, .NET 10 SDK, `PacketProcessor.dll` 을 출력 폴더에 복사
빌드	`dotnet build` (Release 모드 권장)
실행	`SimpleMeter.exe` — 기본 포트 13328 `SimpleMeter.exe 9000` — 포트 직접 지정
키 조작	R 리셋 / Q 종료

PacketProcessor.dll · API Reference · 인쇄: Ctrl+P → PDF로 저장